

Chương 11: Huấn luyện Mạng Nơ-ron Sâu

Giảng viên: [Tên Giảng Viên]

Đại Học Đông Á

Ngày 11 tháng 8 năm 2026

Nội dung Chương 11

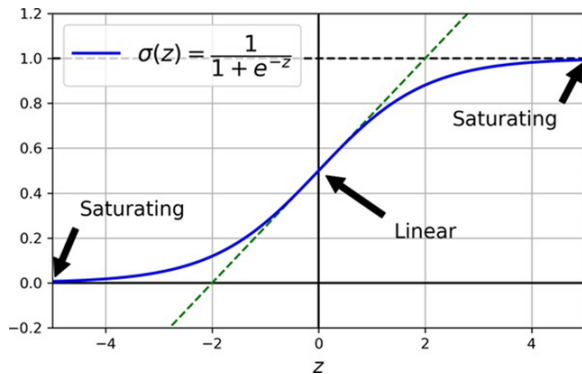
1. Vấn Đề Gradient Biến Mất / Bùng Nổ
2. Các Hàm Kích Hoạt Nâng Cao
3. Batch Normalization và Gradient Clipping
4. Tái Sử Dụng Các Lớp (Transfer Learning)
5. Các Trình Tối Ưu Hóa Tốc Độ Học (Fast Optimizers)
6. Lịch Trình Tốc Độ Học (Learning Rate Scheduling)
7. Kỹ Thuật Chính Quy Hóa (Regularization)

Vấn đề Gradient Biến mất (Vanishing Gradients)

- Thuật toán backpropagation cập nhật các trọng số bằng cách truyền đạo hàm (gradient) từ lớp đầu ra ngược về lớp đầu vào.
- Càng truyền ngược về các lớp sâu hơn (lower layers), gradient thường càng trở nên **nhỏ dần**.
- Hậu quả: Trọng số của các lớp dưới cùng không thay đổi, mạng nơ-ron sâu không thể hội tụ để đạt được kết quả tốt.
- Ngược lại, gradient có thể tăng lên theo cấp số nhân (Exploding gradients), khiến các trọng số lớn đến mức tràn bộ nhớ (thường gặp trong RNN).

Sự Bão Hòa Của Hàm Kích Hoạt Sigmoid

- Vào năm 2010, Glorot và Bengio đã phát hiện ra thủ phạm chính: hàm kích hoạt Sigmoid kết hợp với cách khởi tạo trọng số ngẫu nhiên ban đầu.
- Phương sai của đầu ra mỗi lớp lớn hơn phương sai của đầu vào, làm cho các giá trị ở lớp trên cùng trở nên rất lớn (hoặc rất âm).
- Khi giá trị tuyệt đối lớn, đạo hàm của hàm Sigmoid tiến về 0.



Hình 11-1: Hàm kích hoạt sigmoid bão hòa ở hai đầu

Khởi tạo Trọng số Glorot / He (Initialization)

- Để tín hiệu chảy bình thường trong cả lan truyền xuôi và ngược, **phương sai** của các tín hiệu đầu ra và đầu vào ở mỗi lớp phải **bằng nhau**.
- Xavier Glorot đề xuất cách khởi tạo các trọng số một cách ngẫu nhiên từ phân phối chuẩn với phương sai: $\sigma^2 = \frac{1}{f_{an_{avg}}}$.
- Đối với hàm ReLU, Kaiming He đề xuất khởi tạo **He initialization**: $\sigma^2 = \frac{2}{f_{an_{in}}}$.

Code: Khởi Tạo Trọng Số trong Keras

Theo mặc định, Keras dùng khởi tạo Glorot (với phân phối đều). Để dùng He initialization:

```
import tensorflow as tf
from tensorflow import keras

# Su dung He initialization (thuong ket hop voi ReLU)
dense = keras.layers.Dense(50, activation="relu",
                             kernel_initializer="he_normal")

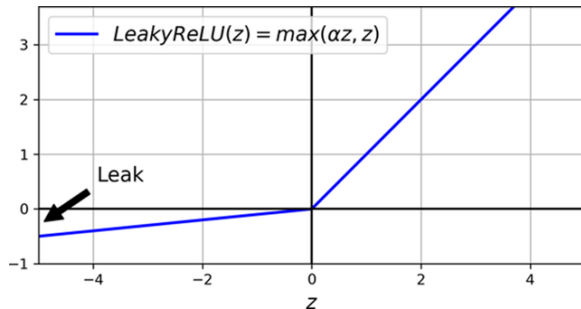
# Hoac su dung phuong sai do LeCun de xuat (ket hop voi SELU)
dense_lecun = keras.layers.Dense(50, activation="selu",
                                   kernel_initializer="lecun_normal")
```

Vấn đề "Dying ReLU"

- Dù hàm ReLU (Rectified Linear Unit) là tiêu chuẩn vàng, nó có một vấn đề gọi là **Dying ReLU**.
- Nếu trọng số của một nơ-ron ReLU được cập nhật sao cho giá trị tổng tĩnh $z < 0$, nó sẽ luôn trả về 0.
- Đạo hàm của hàm ReLU khi $z < 0$ cũng bằng 0, nghĩa là nơ-ron này sẽ "chết" vĩnh viễn và không bao giờ học lại được nữa.

Hàm Leaky ReLU

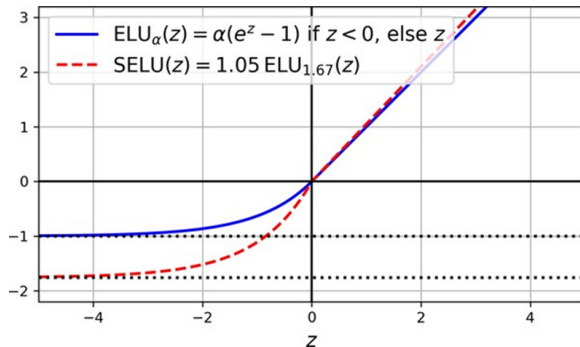
- Giải pháp: $\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$.
- α (thường là 0.01) xác định độ dốc của hàm với các giá trị âm.
- "Độ rò rỉ" (leak) này giúp giữ nơ-ron không bị chết hoàn toàn, tạo cơ hội cho gradient được truyền ngược trở lại.



Hình 11-2: Hàm kích hoạt Leaky ReLU

ELU và SELU

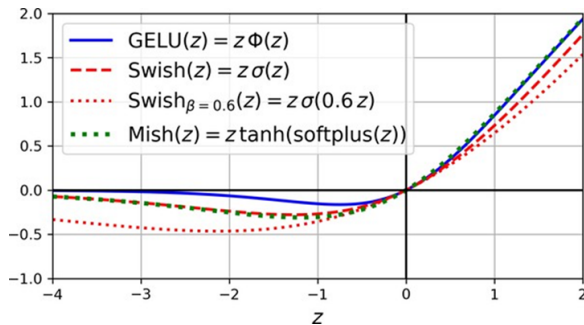
- **ELU (Exponential Linear Unit)**: Mượt mà ở khắp mọi nơi (đạo hàm liên tục tại $z = 0$). Cải thiện tốc độ học tập.
- **SELU (Scaled ELU)**: Nếu tất cả các lớp dày đặc sử dụng SELU, mạng sẽ **tự chuẩn hóa (self-normalize)**, giải quyết vấn đề vanishing gradient.



Hình 11-3: Các hàm kích hoạt ELU và SELU

GELU, Swish, và Mish

- Đây là các hàm phức tạp, mượt mà và không đơn điệu (non-monotonic).
- **GELU** (Gaussian Error Linear Unit) vượt trội trong các mạng NLP (như BERT).
- **Swish** do Google Brain đề xuất, tối ưu hơn trong các bài toán Image Recognition sâu.



Hình 11-4: Các hàm kích hoạt GELU, Swish, và Mish

Batch Normalization (Chuẩn hóa Hàng loạt)

- Khởi tạo He và ELU có thể giảm thiểu vấn đề Vanishing Gradient ở đầu khóa huấn luyện, nhưng nó không đảm bảo không xuất hiện trong quá trình.
- Sergey Ioffe và Christian Szegedy (2015) giới thiệu **Batch Normalization (BN)**.
- Lớp BN chuẩn hóa (tính trung bình và độ lệch chuẩn) của các đầu vào tại mỗi lô (mini-batch) một cách độc lập cho từng tính năng.
- Sau đó, nó áp dụng thay đổi tỷ lệ (scale) γ và độ dịch chuyển (shift) β được học qua backpropagation.

Code: Thêm Batch Normalization trong Keras

- Lớp BN thường được thêm trước hoặc sau Hàm kích hoạt của lớp Ẩn.
- Kinh nghiệm thực tế cho thấy đặt nó TRƯỚC hàm kích hoạt sẽ hoạt động tốt nhất.

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, use_bias=False), # không cần bias vì BN có thuộc tính dịch
        chuyển(beta)
    keras.layers.BatchNormalization(),
    keras.layers.Activation("relu"),
    keras.layers.Dense(100, use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("relu"),
    keras.layers.Dense(10, activation="softmax")
])
```

Hoạt động của BN lúc Inference (Dự đoán)

- Trong lúc Huấn luyện, BN tính mean, variance của một **mini-batch**.
- Khi Đưa vào ứng dụng (Inference), dữ liệu đến từng cái một, làm sao tính mean và variance?
- Giải pháp: BN theo dõi **moving average** (trung bình động) của mean và variance trên toàn bộ tập huấn luyện. Keras sẽ tự động sử dụng giá trị này khi bạn gọi 'model.predict()'.

Gradient Clipping

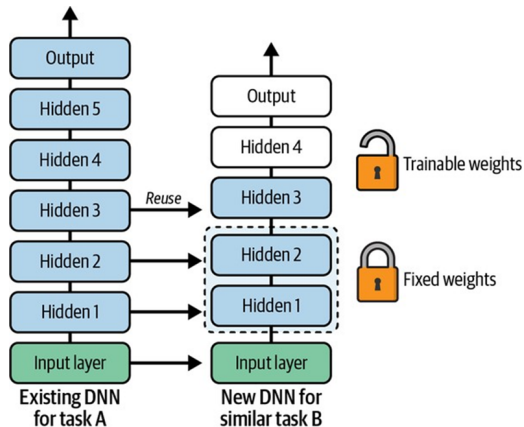
- Một kỹ thuật phổ biến để giảm bớt vấn đề **Exploding Gradients** là Cắt (clip) các gradient để chúng không bao giờ vượt quá một ngưỡng (Threshold).
- Thường được dùng mạnh mẽ trong Mạng Nơ-ron Hồi quy (RNN).

```
# Cắt gradient theo giá trị tuyệt đối (-1.0 đến 1.0)
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)

# Hoặc cắt theo chuẩn (Norm) để không thay đổi hướng của vector
optimizer = keras.optimizers.SGD(clipnorm=1.0)
```

Học Chuyển Giao (Transfer Learning)

- Sẽ rất lãng phí nếu đào tạo mạng CNN siêu lớn từ con số 0.
- Thay vì vậy, ta có thể tìm một Mạng Nơ-ron hiện có thực hiện nhiệm vụ **tương tự**, giữ lại các lớp dưới (Lower layers), và chỉ thay đổi lớp trên cùng (Output layers).
- **Hình 11-5:** Tái sử dụng các lớp (Freeze) giúp tăng tốc độ đáng kể.



Hình 11-5: Tái sử dụng các lớp đã được huấn luyện trước

Code: Tái sử dụng mô hình trong Keras

```
# Tai mot mo hinh da huan luyen (Model A)
model_A = keras.models.load_model("my_model_A.h5")

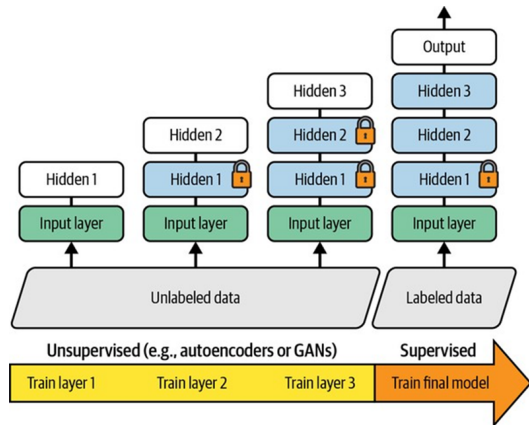
# Tao Model B dua tren cac lop cua Model A (tru lop cuoi)
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))

# Dong bang (Freeze) cac lop cu
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

# Huan luyen vai epoch de cap nhat lop cuoi
model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd")
model_B_on_A.fit(X_train_B, y_train_B, epochs=4)
```

Huấn Luyện Không Giám Sát (Unsupervised Pretraining)

- Điều gì xảy ra nếu bạn có quá ít dữ liệu gán nhãn (Labeled data)?
- Bạn có thể đào tạo trước bằng Học không giám sát (như Autoencoders hoặc RBMs).
- Sau đó chèn thêm lớp phân loại và huấn luyện có giám sát.
- **Hình 11-6:** Các mô hình NLP ngôn ngữ lớn như GPT và BERT cũng là một dạng Unsupervised Pretraining khổng lồ!



Hình 11-6: Học chuyển giao từ huấn luyện không giám sát

Giới thiệu các Optimizer (Trình tối ưu hóa)

- SGD (Stochastic Gradient Descent) cơ bản hoạt động ổn định nhưng **cực kỳ chậm**.
- Các nhà khoa học đã phát triển ra các thuật toán Tối ưu hóa có khả năng chạy nhanh gấp 10-100 lần.
- Các Optimizer nổi tiếng:
 - ➊ Momentum Optimization
 - ➋ Nesterov Accelerated Gradient (NAG)
 - ➌ AdaGrad
 - ➍ RMSProp
 - ➎ Adam / Nadam

Tối ưu hóa Bằng Động Lượng (Momentum)

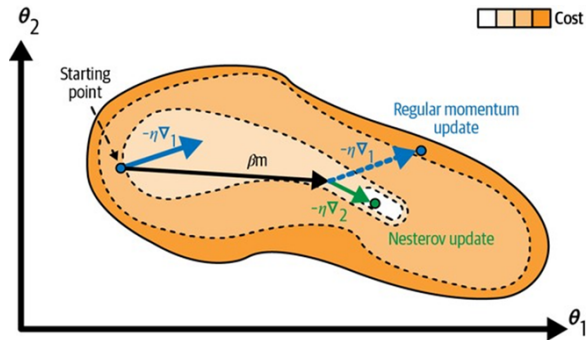
- SGD cập nhật trọng số dựa hoàn toàn trên **gradient tại thời điểm hiện tại**. Nó giống như đi xuống núi nhưng rẽ ngoặt theo góc dốc ngay dưới chân.
- Momentum quan tâm đến **các gradient trước đó**. Giống như thả một quả bóng bowling xuống bề mặt trơn: quả bóng sẽ liên tục **tăng tốc** (động lượng lớn).
- Cập nhật công thức:

$$\begin{aligned}m &\leftarrow \beta m - \eta \nabla_{\theta} J(\theta) \\ \theta &\leftarrow \theta + m\end{aligned}$$

- Keras: `keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)`

Nesterov Accelerated Gradient (NAG)

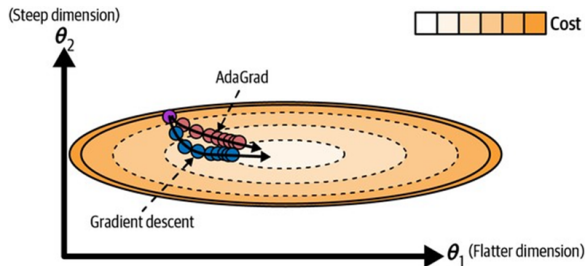
- Biểu thể của Momentum.
- Thay vì tính đạo hàm tại vị trí hiện tại θ , NAG tính đạo hàm ở bước **tiến tới (Look-ahead)** một khoảng βm .
- Điều này giúp thuật toán "đạp phanh" kịp thời khi đến gần đáy lũng.
- **Hình 11-7** so sánh Momentum thông thường (màu xanh lam) và NAG (màu đỏ).



Hình 11-7: So sánh Momentum thông thường và Nesterov

AdaGrad

- Trong các bài toán có nhiều chiều (dimensions) bị thon dài (elongated bowl).
- Thuật toán AdaGrad tự động **làm giảm tốc độ học** dọc theo các chiều có độ dốc quá cao (steep dimensions).
- **Hình 11-8:** AdaGrad đi thẳng tới đích rành mạch hơn thay vì "ngồi ngò" như SGD.
- Lưu ý: AdaGrad bị dừng quá sớm khi dùng cho Neural Networks, do vậy chỉ dùng trong hồi quy tuyến tính.



Hình 11-8: So sánh AdaGrad và Gradient Descent

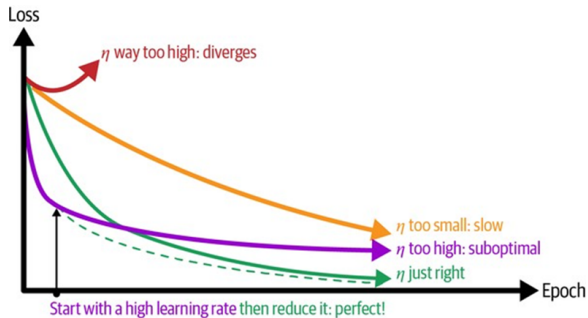
RMSProp và Adam

- **RMSProp**: Sửa chữa nhược điểm của AdaGrad bằng cách chỉ xem xét các gradient gần đây (thay vì tích lũy từ đầu). Rất phổ biến trước khi Adam ra đời.
- **Adam (Adaptive Moment Estimation)**: Kết hợp tính chất tuyệt vời của Momentum (tích lũy gradient) và RMSProp (tích lũy bình phương gradient).
- Adam là Optimizer "mặc định" và thành công vang dội nhất hiện nay cho Mạng Nơ-ron.

```
# Hiện tại Optimizer xin so và đề dùng nhất là Adam
optimizer = keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)
```

Vấn đề của Tốc Độ Học Cố Định

- Tốc độ học (Learning Rate - η) là siêu tham số quan trọng nhất.
- Quá lớn: mô hình phân kỳ. Quá nhỏ: mất quá nhiều thời gian hội tụ.
- **Hình 11-9:** Nếu ta bắt đầu lớn (để tiến nhanh), sau đó giảm dần (để hội tụ chính xác), ta sẽ đạt kết quả tối ưu nhất trong thời gian ngắn nhất!



Hình 11-9: Lịch trình tốc độ học

Code: Power Scheduling trong Keras

- Lập lịch theo hàm lũy thừa: $\eta(t) = \eta_0 / (1 + t/s)^c$.
- Keras thực hiện cực kì dễ thông qua tham số 'decay' (Keras phiên bản cũ) hoặc 'InverseTimeDecay'.

```
lr_schedule = keras.optimizers.schedules.InverseTimeDecay(  
    initial_learning_rate=0.01,  
    decay_steps=10000,  
    decay_rate=1.0)  
  
optimizer = keras.optimizers.SGD(learning_rate=lr_schedule)
```

Code: Performance Scheduling

- Giảm tốc độ học một nửa khi hàm lỗi (validation error) ngưng giảm.
- Trong Keras, sử dụng Callback ReduceLROnPlateau.

```
lr_scheduler = keras.callbacks.ReduceLROnPlateau(  
    factor=0.5, # giảm một nửa  
    patience=5) # nếu sau 5 epoch không giảm lỗi, thì giảm LR  
  
model.fit(X_train, y_train, epochs=50,  
        validation_data=(X_valid, y_valid),  
        callbacks=[lr_scheduler])
```

Tổng Quan Các Phương Pháp Chính Quy Hóa

- "Với hàng tỷ trọng số tự do, mạng nơ-ron sâu có thể học rất tốt, thậm chí nhớ luôn cả nhiều trong dữ liệu."
- Đây chính là **Overfitting** (Quá khớp).
- Các kỹ thuật phổ biến để khắc phục:
 - 1 L1 và L2 Regularization (Tìm hãm trọng số lớn)
 - 2 Dropout (Loại bỏ ngẫu nhiên)
 - 3 Max-Norm (Giới hạn chuẩn vector trọng số)
 - 4 Data Augmentation (Tăng cường dữ liệu bằng biến đổi hình ảnh/âm thanh)

L1 và L2 Regularization

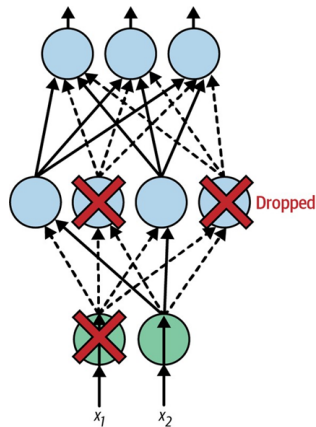
- Cộng thêm hình phạt (penalty) bằng độ lớn của trọng số (Weights) vào hàm Mất Mát (Loss Function).
- Có thể thiết lập dễ dàng trong Keras layers:

```
# Áp dụng l2 regularization với hệ số 0.01
layer = keras.layers.Dense(100, activation="elu",
                             kernel_initializer="he_normal",
                             kernel_regularizer=keras.regularizers.l2(0.01))
```

- Để mã sạch hơn (do lặp lại nhiều lần), bạn có thể dùng tính năng 'functools.partial'.

Dropout - "Liều Thuốc Mê" Cho Nơ-ron

- G. E. Hinton (2012) giới thiệu thuật toán đơn giản một cách vô lý nhưng cực kỳ thành công: **Dropout**.
- Tại mỗi lượt huấn luyện, mỗi nơ-ron có xác suất p (thường từ 10% đến 50%) bị "ngắt" tạm thời.
- Các nơ-ron không thể phụ thuộc vào đồng nghiệp kế bên mà tự thân vận động trích xuất các đặc trưng hữu ích nhất.
- **Hình 11-10:** Minh họa mạng áp dụng kỹ thuật Dropout.



Hình 11-10: Với chính quy hóa dropout, một số nơ-ron bị tắt ngẫu nhiên

Sử dụng Dropout trong Keras

- **Quan trọng:** Khi Dropout huấn luyện, chỉ còn khoảng một nửa số nơ-ron tham gia dự đoán. Vì vậy, trong giai đoạn **Test/Inference**, ta PHẢI nhân giá trị với xác suất giữ lại để cân bằng. (Keras tự động xử lý việc này).

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2), # ngat 20% nơ-ron
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
```

Monte Carlo (MC) Dropout

- Y. Gal và Z. Ghahramani (2016) chỉ ra rằng mạng nơ-ron sử dụng Dropout tương đương toán học với quá trình xấp xỉ suy luận Bayes (Bayesian inference).
- Cụ thể, **MC Dropout** bật Dropout ngay cả trong giai đoạn Inference (Dự đoán).
- Khi ta thực hiện dự đoán 100 lần trên cùng 1 mẫu, ta sẽ nhận được 100 kết quả dự đoán (xác suất) khác nhau do mỗi lần mạng lại tắt đi một số nơ-ron ngẫu nhiên.
- Tính trung bình và phương sai của các dự đoán đó sẽ cho ta một độ tin cậy vô cùng chính xác!

Code: Thực Hiện MC Dropout Bằng Keras

```
import numpy as np

# Bat tinh nang Training = True ben trong lop Dropout ke ca khi du doan
y_probas = np.stack([model(X_test_scaled, training=True)
                     for sample in range(100)])
y_proba = y_probas.mean(axis=0)
y_std = y_probas.std(axis=0) # Do bat dinh (Uncertainty)
```

Alpha Dropout

- Nếu bạn sử dụng hàm kích hoạt **SELU** để mạng tự chuẩn hóa (self-normalize).
- Thì sử dụng Dropout thông thường sẽ phá hủy sự tự chuẩn hóa đó!
- Hãy sử dụng AlphaDropout - một biến thể của Dropout bảo toàn được giá trị trung bình và phương sai.

```
model.add(keras.layers.AlphaDropout(rate=0.2))
```

Max-Norm Regularization

- Hạn chế (clamp) không cho chuẩn l2 (Norm 2) của vector trọng số của mỗi nơ-ron vượt quá một giá trị lớn nhất r .
- Cụ thể: $\|\mathbf{w}\|_2 \leq r$.
- Nếu vượt quá, trọng số sẽ được chia tỉ lệ: $\mathbf{w} \leftarrow \mathbf{w} \frac{r}{\|\mathbf{w}\|_2}$.
- Max-norm kết hợp rất tốt với Dropout để kìm hãm sự tăng trưởng vô hạn của các trọng số.

Code: Max-Norm Trong Keras

```
dense = keras.layers.Dense(100, activation="elu",  
                             kernel_initializer="he_normal",  
                             kernel_constraint=keras.constraints.max_norm(1.))
```

Tối ưu hóa RMSProp

- AdaGrad đôi khi giảm tốc độ học quá mức và dừng sớm trước khi đến mục tiêu cực trị toàn cục (global optimum).
- RMSProp là phiên bản nâng cấp của AdaGrad, giải quyết việc dừng sớm bằng cách chỉ tích lũy bình phương của các gradient **gần đây nhất** thay vì tất cả gradient từ đầu thuật toán.
- Thuật toán này sử dụng tốc độ phân rã lũy thừa β (exponential decay).

Code: Tối Ưu Hóa RMSProp Keras

```
# Tinh chỉnh tốc độ học bằng RMSprop  
optimizer = keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

- Tham số 'rho' tương ứng với β trong công thức lý thuyết (thường đặt là 0.9).
- Ngoại trừ những kiến trúc cực kỳ phức tạp, RMSProp gần như luôn hoạt động tốt hơn hẳn so với AdaGrad.

- **Adam** kết hợp Momentum (theo dõi động lượng) và RMSProp (theo dõi bình phương gradient).
- **Nadam** là Adam cộng với kỹ thuật Nesterov Accelerated Gradient (NAG), cho tốc độ hội tụ nhanh hơn một chút.
- **AdamW** là Adam kết hợp với kỹ thuật điều chuẩn phân rã trọng số (weight decay regularization) thực thụ, thường dùng cho mạng biến áp (Transformers).

Tóm Tắt Lựa Chọn Các Kỹ Thuật (Phần 1)

Thành phần	Lựa chọn mặc định
Cấu hình khởi tạo (Init)	Khởi tạo He (He initialization)
Hàm kích hoạt (Activation)	ELU hoặc GELU (hoặc Swish)
Chuẩn hóa (Normalization)	Mạng nông/sâu: Batch Normalization (BN)
Chống quá khớp (Regularize)	Early Stopping + Dropout
Trình tối ưu hóa (Optimizer)	Nadam hoặc AdamW (kèm Warmup)
Lịch trình tốc độ học	Performance Scheduling (hoặc One Cycle)

Tóm Tắt Lựa Chọn Nếu Tự Chuẩn Hóa (Phần 2)

Nếu bạn xây dựng Mạng dày đặc (Dense) và muốn nó tự chuẩn hóa (Self-Normalizing):

Thành phần	Cấu hình Mạng Tự Chuẩn Hóa
Cấu hình khởi tạo (Init)	Khởi tạo LeCun
Hàm kích hoạt (Activation)	SELU
Chuẩn hóa (Normalization)	Không cần (tự thân chuẩn hóa)
Chống quá khớp (Regularize)	Alpha Dropout

Các Vấn Đề Thực Tế Khác

- Nếu bạn làm việc trên các mô hình Thưa (Sparse models), hãy dùng L_1 Regularization.
- Nếu thiết bị của bạn cần tốc độ phản hồi cực nhanh (low latency) khi ứng dụng thực tế (Inference), hãy loại bỏ Batch Normalization bằng cách hợp nhất (fuse) các tham số vào lớp trước đó, và dùng Leaky ReLU.
- Luôn tận dụng tài nguyên có sẵn: Đừng quên Transfer Learning!

Tổng Kết Chương 11

- Chúng ta đã giải quyết rất ráo việc **Huấn Luyện Các Mạng Khổng Lồ** mà không để rơi vào tình trạng Vanishing Gradient.
- **Khởi tạo He** kết hợp hàm **ELU / GELU** là lựa chọn tiêu chuẩn.
- Củng cố bằng **Batch Normalization**.
- Sử dụng **Adam Optimizer** với Lịch trình Học Tập (Learning Rate Schedule).
- Và đương nhiên, chống Overfitting hiệu quả bằng **Dropout**.